

Docker Practical Guide– Docker Images vs Docker files CLI Basics

Part 1: Install Docker

Step 1 – Update package list

```
sudo apt update
```

Meaning: Updates Ubuntu's list of available software packages.

Step 2 – Install Docker

```
sudo apt install docker.io -y
```

Meaning: Installs Docker. `-y` automatically answers Yes.

Step 3 – Start Docker

```
sudo systemctl start docker
```

Meaning: Starts the Docker service.

Step 4 – Start Docker automatically

```
sudo systemctl enable docker
```

Meaning: Makes Docker start automatically when Ubuntu boots.

Step 5 – Check Docker version

```
docker --version
```

Meaning: Confirms that Docker is installed.

Step 6 – Check Docker service

```
sudo systemctl status docker
```

Meaning: Checks whether Docker is running.

Press **Q** to exit the status screen.

Part 2: Run Your First Docker Container

Step 7 – Run Hello World

```
sudo docker run hello-world
```

Meaning: Downloads the `hello-world` image if needed and runs a container from it.

Expected: Docker displays a "Hello from Docker!" message.

it means:

- ✓ Docker is installed
- ✓ Docker service is running
- ✓ Docker can download images
- ✓ Docker can create containers
- ✓ Docker can run containers

Optional: Use Docker without `sudo`

```
sudo usermod -aG docker $USER
```

Then log out and log in again.

Part 3: Download and Run Ubuntu Container

Step 8 – Download Ubuntu image

```
sudo docker pull ubuntu
```

Meaning: Downloads the Ubuntu Docker image from Docker Hub.

Step 9 – View Docker images

```
sudo docker images
```

Meaning: Shows images available on your computer.

Step 10 – Run Ubuntu container

```
sudo docker run ubuntu
```

Meaning: Creates and starts a container using the Ubuntu image.

The container exits immediately because there is no long-running command.

Step 11 – Run Ubuntu interactively

sudo docker run -it ubuntu

Meaning:

- `-i` → keeps the terminal interactive
- `-t` → provides a terminal

You are now **inside the Ubuntu container**.

Type:

`exit`

to leave the container.

Part 4: Check and Manage Containers

Step 12 – Show running containers

sudo docker ps

Meaning: Shows currently running containers.

View all containers

sudo docker ps -a

Shows running + stopped containers.

Container ID	Image	Status	Meaning
3ce025914d0b	2260313b31c8	Exited (0)	Stopped normally
7fc544f0d352	2260313b31c8	Exited (127)	Stopped because a command was not found
d6afbcbf6d30	hello-world	Exited (0)	Finished normally

Important: Exited (0) vs Exited (127)

Exited (0) means:

Container finished successfully.

For example, `hello-world` prints its message and then finishes, so it becomes **Exited (0)**.

Exited (127) usually means:

The command that Docker tried to run could not be found.

To delete a Docker image, use:

sudo docker rmi <image_ID>

egs: `sudo docker rmi 5dd0d3e6e255`

if, The image cannot be deleted because a **stopped container** was created from it.

The stopped container is:

Egs : 59d9cff481ff

Step 1 — Delete the stopped container

```
sudo docker rm 59d9cff481ff
```

Step 2 — Delete the image

```
sudo docker rmi 5dd0d3e6e255
```

Step 3 — Check

```
sudo docker images
```

Step 13 – Run a container in background

```
sudo docker run -d ubuntu sleep 5000
```

or

```
sudo docker run -d ubuntu sleep infinity
```

Meaning: Runs Ubuntu in the background for 5000 seconds.

- `-d` → detached/background mode
- `sleep 5000` → keeps the container running

Step 14 – Check running container

```
sudo docker ps
```

Meaning: Displays the running container and its ID.

Step 15 – Stop the container

```
sudo docker stop <container_id>
```

Meaning: Stops the specified container.

Example:

```
sudo docker stop dc452ee284ac
```

Step 16 – Start the container

```
sudo docker start <container_id>
```

Step 17 – Execute the container

```
sudo docker exec -it ba66c9eabcff bash
```

Step 18 – Remove the container

```
sudo docker rm <container_id>
```

Step 19 – Show all containers

```
sudo docker ps -a
```

Step 20 – Remove unwanted containers

```
sudo docker rm <container_id1> <container_id2>
```

Step 21 – Remove all containers

```
sudo docker container prune
```

Step 22 – Remove all Images

```
sudo docker image prune -a
```

Part 5: Run a Web Server Using Docker

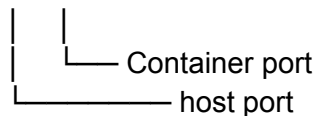
Step 19 – Run Nginx

```
sudo docker run -d -p 8080:80 nginx
```

Meaning: Runs an Nginx web server inside a container.

Important:

-p 8080:80



So:

localhost:8080 → Docker container port 80 → Nginx

Step 20 – Check the container

`sudo docker ps`

Meaning: Confirms that the Nginx container is running.

Step 21 – Open the website

Open a browser and enter:

`http://localhost:8080`

Meaning: Accesses the Nginx web server running inside Docker.

Part 6: Create Your Own Docker Website

Step 22 – Create a project folder

`mkdir docker-demo`

Meaning: Creates a folder for our Docker project.

Step 23 – Enter the folder

`cd docker-demo`

Meaning: Moves into the project directory.

Step 24 – Create the webpage

`nano index.html`

Enter:

```
<!DOCTYPE html>
<html>
<head>
<title>Docker Demo</title>
</head>
<body>
<h1>Hello docker training</h1>
</body>
</html>
```

Save using:

Step 25 – Check the webpage

`cat index.html`

Meaning: Displays the contents of `index.html`.

Part 7: Create a Dockerfile

Step 26 – Create Dockerfile

`nano Dockerfile`

Enter:

```
FROM nginx
COPY index.html /usr/share/nginx/html/
EXPOSE 80
```

What does it mean?

Command	Meaning
<code>FROM nginx</code>	Use Nginx as the base image
<code>COPY index.html /usr/share/nginx/html/</code> <code>...</code>	Copy our webpage into Nginx's website folder
<code>EXPOSE 80</code>	Indicates that the container uses port 80

Save and exit.

Step 27 – Check Dockerfile

`cat Dockerfile`

Meaning: Displays the Dockerfile contents.

Part 8: Build Your Own Docker Image

Step 28 – Build the image

`sudo docker build -t mywebsite .`

For particular docker file,

`sudo docker build -f <Dockerfile-name> -t mywebsite .`

Egs: `sudo docker build -f Dockerfile -t mywebsite .`

Meaning: Creates a Docker image from the Dockerfile.

- `build` → build an image
- `-t mywebsite` → give the image the name `mywebsite`
- `.` → use the current directory as the build context

Step 29 – Check images

`sudo docker images`

Meaning: Confirms that `mywebsite` has been created.

Part 9: Run Your Website Container

Step 30 – Run the website

`sudo docker run -d -p 8080:80 mywebsite`

Meaning: Creates and runs a container from our `mywebsite` image.

Step 31 – Check running containers

`sudo docker ps`

Meaning: Confirms that the website container is running.

Step 32 – Open the website

Open your browser:

`http://localhost:8080`

You should see:

Hello docker training

You have successfully created and hosted your own webpage using Docker!

To share the local webpage, punch the local webpage through a tunnel

cloudflared tunnel --url <http://localhost:8080>

Install cloudflared

1. Download the Cloudflare package

wget

<https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64.deb>

2. Install it

sudo dpkg -i cloudflared-linux-amd64.deb

If it reports missing dependencies, run:

sudo apt --fix-broken install

3. Check installation

cloudflared --version

And share the URL printed in your terminal

egs : <https://random-subdomain.trycloudflare.com>

Important Docker Commands Learned

Command

Purpose

<code>docker --version</code>	Check Docker version
<code>docker pull</code>	Download an image
<code>docker images</code>	List images
<code>docker run</code>	Create and run container
<code>docker ps</code>	Show running containers
<code>docker ps -a</code>	Show all containers
<code>docker stop</code>	Stop container
<code>docker rm</code>	Remove container
<code>docker rmi</code>	Remove image
<code>docker build</code>	Build an image
<code>docker logs</code>	View container output